Micro Hack 1 - Business Apps Workbook

FY27 - Fabric Apps Motion (EMEA)

Contents

1 Micro Hack 1 — Business Apps Workbook (Participants) 1
1.1 Day agenda (1 day) . . . 1

2 Part A — Understand 2
2.1 1) The scenario in plain words . . . 2
2.2 2) What is Rayfin? (60-second primer) . . . 3
2.3 3) The result you are aiming for . . . 3
2.4 Step 0 — Prerequisites & environment setup (do this first) . . . 4

3 Part B — Build it step by step 5
3.1 Step 1 — Create the project (no clone needed) . . . 5
3.2 Step 2 — Run the empty app first (your “it works” check) . . . 6
3.3 Step 3 — Declare the data model . . . 6
3.4 Step 4 — Build the Bicycle Board (your first screen) . . . 7
3.5 Step 5 — Build the Pit-Stop Queue (take action) . . . 7
3.6 Step 6 — Add the Ride Mood & KPI screen . . . 7
3.7 Step 7 — Add the Live Map (make it shine) . . . 8
3.8 Step 8 — Polish (roles & visuals) . . . 8

4 Wrap-up 8
4.1 Team framing (fill this in 2 minutes) . . . 8
4.2 Demo storyboard (5 minutes) . . . 9
4.3 Reflection . . . 9
4.4 Reference assets . . . 9

1 Micro Hack 1 — Business Apps Workbook (Participants)

Track: **End-customer Apps (Rayfin)** Scenario: **Helios Bicycle** (fictional final customer)

Print-ready PDF: `microhack-1-business-apps-workbook.pdf`

This is a **didactic** micro-hack. You don’t need prior Rayfin experience — every step explains **what** you do, **why**, and **what you should see**. Follow them in order.

1.1 Day agenda (1 day)

Morning — presentations & demos. Afternoon — hands-on hack. This is a **standalone one-day micro-hack for the Business Apps (Rayfin) track**.

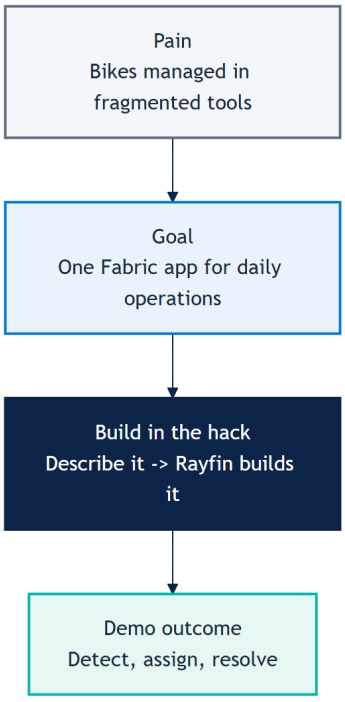
Time	Session
09:00	Welcome & motion overview — the Fabric apps motion for EMEA
09:30	Microsoft Fabric foundations — OneLake, capacity, workspaces
10:00	Rayfin foundations — spec-driven apps, data model = database
10:30	Break
10:45	**Demo · Helios Bicycle Studio** — the app you'll build today
11:15	The hack brief, teams & environment check
12:00	Lunch
13:30	**Hack · Sprint 1** — build the core app (steps 1–5)
15:30	Break
15:45	**Hack · Sprint 2** — extend & polish (steps 6–8)
16:45	Team demos (5 min each)
17:00	Wrap-up, KPIs & next steps

---

2 Part A — Understand

2.1 1) The scenario in plain words

Helios Bicycle runs shared city bikes across several stations. Today the station teams track bikes, repairs and rider feedback in spreadsheets and chat — slow and error-prone. Your job in the afternoon: build **one simple app**, *Helios Bicycle Studio*, that lets a manager see every bike, fix the right one first, and keep riders happy.



2.2 2) What is Rayfin? (60-second primer)

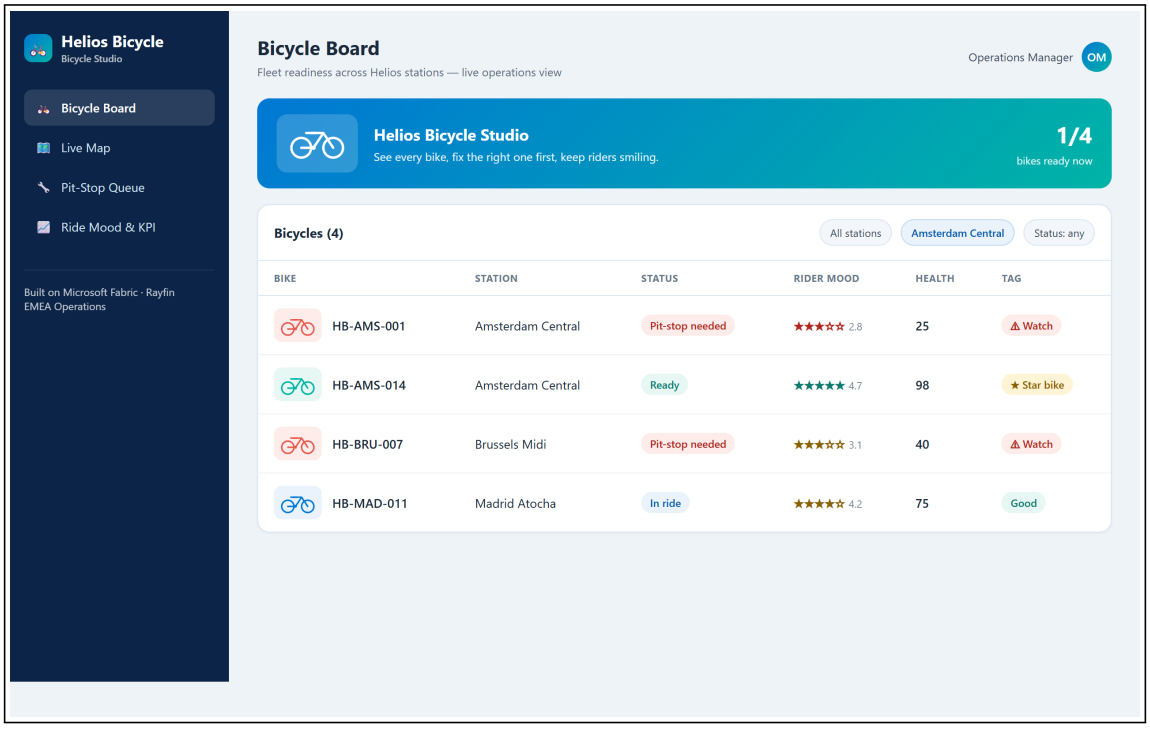
Rayfin is a *Backend-as-a-Service on Microsoft Fabric*. In plain terms:

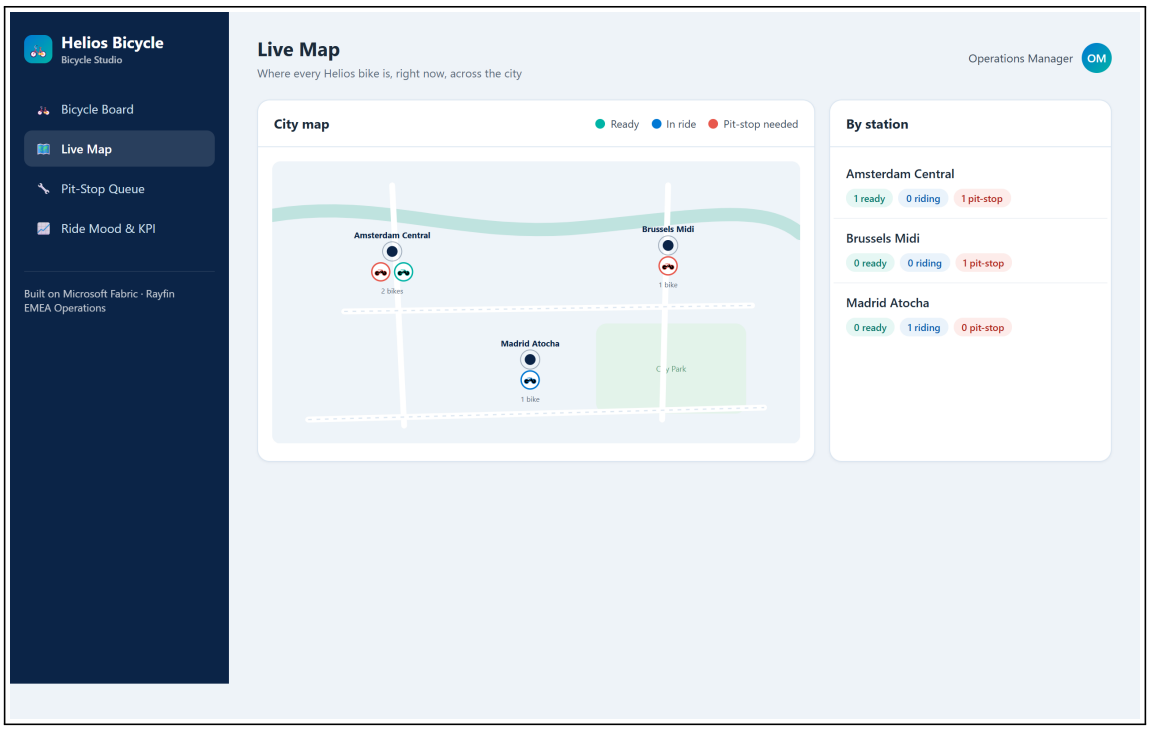
- You **describe your data** (e.g. a `Bicycle` has a code, a station, a status). Rayfin then **creates and manages the database** for you in Fabric — no SQL, no servers to run.
- You **describe your screens in natural language** (with GitHub Copilot). Rayfin generates the app UI against that data.
- So building an app becomes: *declare the data -> describe the screens -> run*.

Key idea to remember: **your data model = your database**. When you declare a `Bicycle` type, Rayfin makes the matching table automatically.

2.3 3) The result you are aiming for

By the end of the afternoon your app should look like this — a **Bicycle Board** and a **Live Map** of the fleet:





2.4 Step 0 — Prerequisites & environment setup (do this first)

Modeled on Microsoft's Build LAB514 setup. On a lab VM most tools are pre-installed.

Tools you need:

- Node.js 20+ (24 recommended) and npm
- Visual Studio Code
- **GitHub Copilot CLI** (the `copilot` terminal agent)

Sign in (two accounts):

1. **GitHub Copilot** — in a terminal run `copilot`, then `/login`, choose **GitHub.com**, finish the browser sign-in, confirm *"Signed in successfully"*, then `/exit`.
2. **Microsoft Fabric** — open `https://app.fabric.microsoft.com` and sign in. *(If it shows "Power BI" bottom-left, switch it to "Fabric".)*

Fabric workspace: create (or use) a workspace with a **Fabric capacity** assigned — this is where your app deploys.

Verify your setup:

```
node --version
npm --version
copilot --version
```

What you should see. All three print a version, and you're signed in to GitHub Copilot and to Fabric with a capacity-backed workspace.

3 Part B — Build it step by step

How to work. There is no repository to clone for this track — Rayfin scaffolds the whole project for you with one command (Step 1). After that you build the app **conversationally** with the **GitHub Copilot CLI** (`copilot` in your terminal): you give it a short prompt, it writes the code, and you check the result in the browser. Each step below = one action + **what you should see**.

3.0.1 Who does what — Copilot vs. the Rayfin CLI (read once)

This is the key thing to understand: **the prompt + Copilot do NOT deploy anything by themselves.**

- **GitHub Copilot** = writes/edits your TypeScript: the data model in `rayfin/data/` and the screens. It does **not** touch Fabric or your database.
- **The Rayfin CLI** = *you* run it. It deploys to Fabric, **provisions the database from your data model**, and hosts the app. `npm run dev` wraps this for the dev loop.

So the rhythm is: *ask Copilot to write code -> run a CLI command to deploy/provision -> look at the app.*

Command	What it does
`npm run dev`	Dev loop: deploys (`rayfin up`) + serves the app for fast iteration. **Use this most of the time.**
`npx rayfin up`	Full deploy to Fabric: creates the app item, applies the DB schema from your data model, builds & uploads the UI, prints the live URL.
`npx rayfin up db apply`	After you change `rayfin/data/`, push only the new schema (add `--force` if it warns about data loss).
`npx rayfin up staticapp deploy`	Redeploy only the frontend (faster).
`npx rayfin up status`	Show the current deployment.
`npx rayfin up --dry-run`	Preview a deploy without changing anything.
`npx rayfin login`	Re-authenticate if you hit a 401/403.

Auth note. Email/password sign-in only works in local dev. Once deployed to Fabric, only **Fabric brokered auth (Entra SSO)** works — `rayfin.yml` must have `auth.fabric.enabled: true`.

3.1 Step 1 — Create the project (no clone needed)

What you do. Scaffold a Rayfin project. “Scaffold” means *generate a ready-to-run starter project* — you do **not** clone any repo.

```
npm create @microsoft/rayfin@latest -- --template dataapp
cd <the-folder-it-created>
```

> **Pick a valid template.** `dataapp` is the recommended base (a Fabric-authenticated React + Vite app wired for Rayfin data — you add entities in `rayfin/data/`). Running `npm create @microsoft/rayfin@latest` with no `--template` lets you choose interactively from `blankapp`, `dataapp`, `gettingstartedauth`, `todoapp`.

Why. Rayfin gives you a complete app skeleton (data layer + UI) so you start from something that already runs.

What you should see. A new project folder is created and dependencies install.

3.2 Step 2 — Run the empty app first (your "it works" check)

What you do. Start the starter app **before changing anything** — exactly like a Hello World, to confirm your environment and Fabric connection are healthy.

```
npm run dev          # deploys to Fabric and provisions the database, then
 ↪  opens the app
```

Why. `npm run dev` pushes the project to Fabric and **provisions your database** from the data model in `rayfin/data/*.ts`. If the starter app opens, your whole chain works and you can safely start customizing.

What you should see. The starter app opens in the browser and your Fabric workspace shows the new app. **Don't go further until this works.**

3.3 Step 3 — Declare the data model

What you do. Tell Copilot the data your app is about. This defines the **database tables**.

```
Build "Helios Bicycle Studio", a fun bike-operations app on Rayfin.
Data model:
- Bicycle(bikeCode, station, status[ready|in-ride|pit-stop-needed],
 ↪  featured)
- RideSession(bicycle, riderAlias, moodScore, startedAt, endedAt)
- PitStopTicket(ticketCode, bicycle, station, priority[high|normal],
                status[new|assigned|done], issue, openedAt,
 ↪  assignedMechanic)
- MechanicProfile(displayName, station, active)
Roles: Operations Manager has full access; Mechanic sees only assigned
 ↪  tickets.
Style: clean, friendly, Microsoft Fluent, blue #0078d4 + teal #00b4a6.
```

Why. These four types become four tables. `Bicycle.status` is a fixed list of values, so the app can colour bikes by status later.

What you should see. Copilot creates the data files; running the app re-provisions the database with the new tables (still empty screens — that's normal).

3.4 Step 4 — Build the Bicycle Board (your first screen)

What you do. Ask for the main screen: the list of bikes a manager looks at every morning.

```
Add a "Bicycle Board" screen: a table grouped by station showing a status
↪ pill,
rider mood as stars, a health score and a tag (Star / Good / Watch).
Add filters for station and status.
```

Why. This is the app's home base — *detect* which bikes need attention. The health score/tag is computed from status + mood so the manager sees priorities at a glance.

What you should see. A table of bikes by station with coloured status pills and a tag. You can filter by station and status. (Compare with the first example image above.)

3.5 Step 5 — Build the Pit-Stop Queue (take action)

What you do. Add the screen where a manager turns a problem into an assigned repair.

```
Add a "Pit-Stop Queue" screen as a kanban with columns new / assigned /
↪ done.
Let me create a ticket from a bike, and assign a mechanic in one click,
preferring the same-station and least-loaded mechanic.
```

Why. This is the *assign -> resolve* half of the story. "One click" assignment keeps the manager fast; preferring the same-station, least-busy mechanic is a simple, sensible rule.

What you should see. A three-column board. Creating a ticket from a bike places a card in **New**; assigning moves it to **Assigned** with the mechanic's name.

> **Milestone (Sprint 1):** Steps 1–5 give you a working app — detect a bike, open a ticket, assign it. If you reach here, you can already demo.

3.6 Step 6 — Add the Ride Mood & KPI screen

What you do. Add a light analytics screen.

```
Add a "Ride Mood & KPI" screen with cards for average rider mood,
number of bikes needing a pit-stop, and a workshop -> PoC -> opportunity
↪ funnel.
```

Why. Rider mood is an early quality signal; the funnel cards connect the app to the **business KPI** of the motion (workshops turning into opportunities).

What you should see. A row of KPI cards with the average mood, the count of bikes needing a pit-stop, and the funnel percentages.

3.7 Step 7 — Add the Live Map (make it shine)

What you do. Add a map view of the fleet.

```
Add a "Live Map" screen: a stylized city map with one marker per bike
 ↪  colored by
status (teal = ready, blue = in ride, red = pit-stop needed), plus a
 ↪  per-station
summary panel.
```

Why. A map turns rows of data into an instantly readable picture — great for the demo and close to how operations teams think about a city fleet.

What you should see. A map with coloured bike markers and a side panel counting bikes per station (compare with the second example image above).

3.8 Step 8 — Polish (roles & visuals)

What you do. Tighten the experience. Pick what you have time for.

```
Make the bike rows more visual: add a colored bike illustration whose color
reflects the status (teal = ready, blue = in ride, red = pit-stop needed).
```

```
Enforce roles: a Mechanic user should only see pit-stop tickets assigned
 ↪  to them.
```

Why. Visual cues speed up reading; role rules make the app realistic for a real customer rollout.

What you should see. Coloured bike icons on each row, and a mechanic account that only sees its own tickets.

> **Milestone (Sprint 2):** the app is graphical and role-aware — ready for the 5-minute demo.

---

4 Wrap-up

4.1 Team framing (fill this in 2 minutes)

- Team name:
- Who is the primary user (manager or mechanic)?
- One sentence: what does your app make faster?

4.2 Demo storyboard (5 minutes)

1. **Context** (30s): who is the user, what hurts today.
2. **Flow** (3m): detect on the Board -> open a ticket -> assign on the Queue -> show the Map.
3. **Architecture** (45s): how the data and the app connect in Fabric (Rayfin provisioned it).
4. **Next step** (45s): what you'd add before a real rollout.

4.3 Reflection

- What was the easiest part of describing the app to Rayfin?
- What did you have to clarify for Copilot to get it right?
- What metric would prove value in week 1 of production?

4.4 Reference assets

- Reference solution code: `src/apprayfin/`
- Rayfin prompt baseline (canonical spec): `src/apprayfin/src/specs/helios-bicycle-app-spec.md`
- Trainer answer key + screenshots: `docs/microhack-1-business-apps-solution.md`